

PORTFOLIO 2026

화면에서 시작해 서버와 데이터까지

처음 HTML을 배우던 학부 캡스톤에서,
두 번의 팀 프로젝트 1위와 백엔드 설계까지.

이동근

Frontend Developer

E-mail mark6896@naver.com

Mobile 010-9950-6896

GitHub github.com/dongkeun99

모르는 자리에서 가장 먼저 손을 드는 개발자 이동근입니다. 기계공학에서 출발해 스스로 진로를 바꿨고, 그렇게 자원한 자리에서 HTML을 처음 배웠습니다. 부트캠프 두 프로젝트에서 1위를 했으며, 지금은 백엔드를 맡고 있습니다. 화면만 만드는 개발자보다 데이터가 어디서 와서 어떻게 흐르는지 아는 개발자가 되려 합니다.

1위

부트캠프 팀 프로젝트
2회 모두 평가 1위

4인

부트캠프 우수상
(전체 수강생 중 선정)

동상

학부 캡스톤 디자인
(첫 개발 프로젝트)

• 낯선 스택을 학습해 온 이력

○ 2023.03

원광대학교 컴퓨터·소프트웨어공학과 편입

기계자동차공학과에서 진로를 바꿔 3학년으로 편입. **2년치 전공을 독학으로 따라잡으며** 평점 3.61로 졸업.

○ 2023.09

첫 캡스톤 디자인 — HTML을 처음 만지다 · 동상

팀에서 **아무도 말지 않으려던 개발을 자원**. 강의 영상을 뒤져가며 HTML·CSS를 익혀 화면을 완성했고, 이때 프론트엔드 개발자를 목표로 정함.

○ 2024

Flutter 첫 접촉 — 팀장으로 안드로이드 앱 완주

새 언어와 새 플랫폼을 동시에 익히며 팀장을 맡아 앱을 완성. 웹 바깥의 개발 환경을 처음 경험.

○ 2025

정보처리기사 취득

소프트웨어 설계·개발, 데이터베이스 구축 등 전공 지식을 자격 취득으로 정리.

○ 2026.01 - 07

멋쟁이사자처럼 프론트엔드 부트캠프 · 1위 2회 · 우수상

JavaScript · TypeScript · React · Next.js를 **6개월 만에 실무 수준으로 끌어올림**. 중간·파이널 두 팀 프로젝트에서 모두 평가 1위, 전체 수강생 중 4인에게 주어지는 우수상 수상.

○ 2026.06 -

해커톤 참여 — 프론트엔드 개발자가 백엔드를 자원

파이널 프로젝트에서 Supabase를 다루며 화면 너머를 알아야 서비스가 보인다고 느껴, **익숙한 프론트엔드 대신 백엔드를 자원**.

Frontend

TypeScript, JavaScript(ES6+)

React, Next.js (App Router)

HTML/CSS, Tailwind CSS

서버·클라이언트 컴포넌트를 나누어 초기 로딩을 줄이고, 데스크톱·모바일이 다른 형태로 전환되는 반응형 UI를 구현합니다.

Backend / Data

Supabase (PostgreSQL)

인증 연동, 테이블 설계

Realtime 구독, DB Trigger

정합성이 중요한 값은 애플리케이션이 아니라 DB가 책임지도록 트리거로 처리한 경험이 있습니다.

설계 · 추상화

공통 컴포넌트 인터페이스 설계

관심사 분리 (표시 / 상태)

테이블 분리 및 2단계 저장 구조

여러 화면에서 반복되는 구조를 찾아, 컴포넌트가 **몰라도 되는** 것을 밖으로 밀어내는 방식으로 설계합니다. (6페이지)

협업 / 환경

Git · GitHub, CLI

Vercel 배포

Figma 기반 화면 설계 협업

브랜치를 나눠 기능 단위로 작업하고, 팀 컨벤션(서비스 함수·클라이언트 컴포넌트 분리)에 맞춰 개발합니다.

• 자격 및 학력

정보처리기사

한국산업인력공단 · 2025 취득

소프트웨어 설계·개발, 데이터베이스 구축

원광대학교

컴퓨터·소프트웨어공학과 · 2025.08 졸업

3학년 편입 · 평점 3.61

기술을 나열하기보다, **왜 그 선택을 했는지 설명할 수 있는 것만** 적었습니다. 다음 장부터는 위 기술이 실제 프로젝트에서 어떤 문제를 만나 어떻게 쓰였는지를 정리했습니다.

필리핀 거주자를 위한 **생활 필수 플랫폼**입니다. 중고거래 · 구인구직 · 부동산을 한 곳에서 다루며, 저는 **회원가입, 중고물품 상세, 관리자 회원 관리, 부동산 등록, 공통 컴포넌트 설계**를 맡았습니다. 부트캠프 파이널 프로젝트 평가에서 **전체 팀 중 1위**를 기록했습니다.

기간 2026.05 ~ 2026.07 (6주)

팀 구성 프론트엔드 4인

담당 회원가입 · 중고물품 상세 · 관리자 · 부동산 · 공통 컴포넌트

Next.js (App Router)

TypeScript

Tailwind CSS

Supabase

PostgreSQL

Realtime

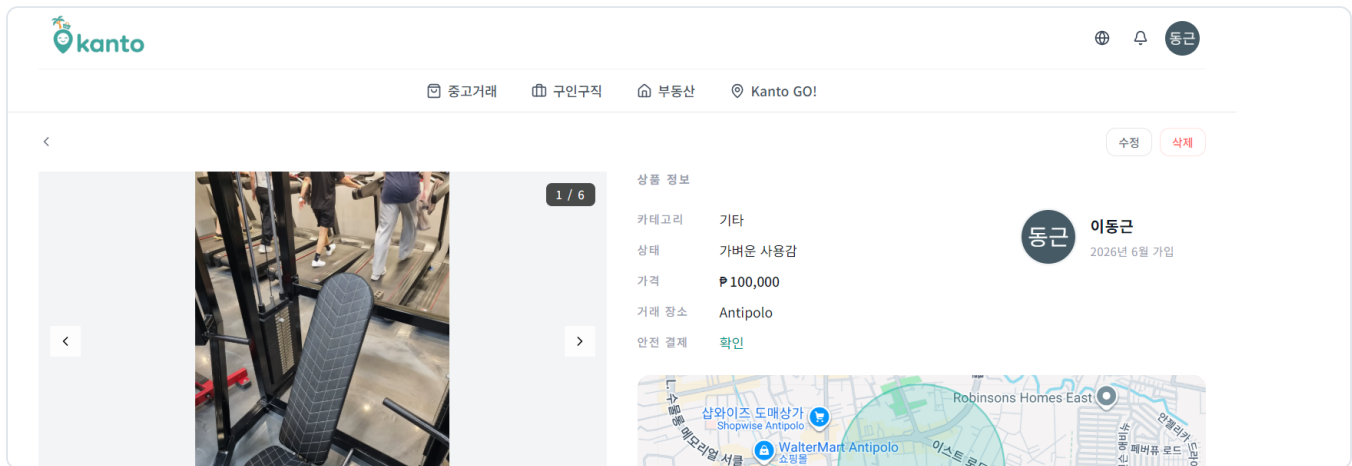
DB Trigger

Vercel

DEMO kanto-iota.vercel.app

CODE github.com/FrontendBootcamp-17th/kanto

서비스 화면

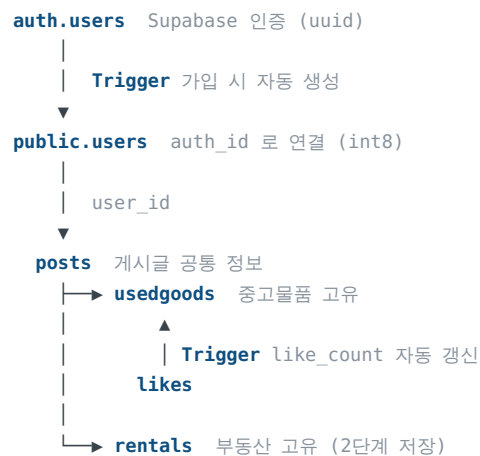


중고물품 상세 — 이미지 캐러셀, 거래 지역 지도, 작성자 전용 수정·삭제

회원가입

회원가입 — 약관 동의 및 유효성 검증

담당 파트의 데이터 구조



회원가입 /signup

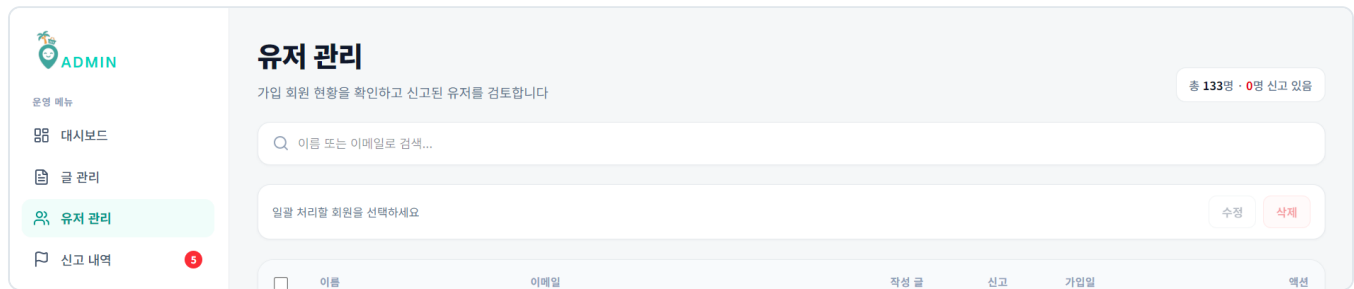
- Next.js App Router 기반 폼 상태 관리 및 입력 **유효성 검증**, 필수·선택 약관 동의 UI
- Supabase **인증(auth) 연동** 및 가입 시 서비스 회원 정보 자동 생성 **DB Trigger** 작성 (문제 해결 02)

중고물품 상세 페이지 /usedgoods/[id]

- 다중 이미지 **캐러셀** 및 썸네일 네비게이션, 거래 지역 지도 연동
- Supabase **Realtime 구독**을 활용한 실시간 좋아요 — 다른 사용자의 반응이 새로그침 없이 반영
- DB Trigger**로 좋아요 수 자동 집계 (문제 해결 01)
- 신고 modal, 작성자 전용 **수정·삭제** 기능 및 수정 페이지 구현 — **이후 팀 공통 컴포넌트로 확산 (6페이지)**

관리자 회원 관리 /admin/users, /admin/users/[id]

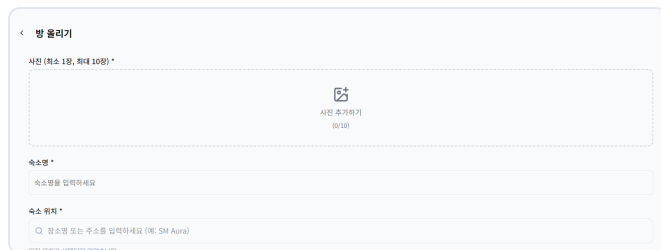
- 회원 목록·상세 화면, 검색 및 일괄 처리 기능
- 서버 컴포넌트로 데이터 조회 / 클라이언트 컴포넌트로 상호작용**을 분리해 초기 로딩 최적화
- 데스크톱은 테이블형, 모바일은 카드형으로 전환되는 **반응형 UI**



관리자 — 회원 목록, 검색 및 일괄 처리

부동산 등록 /rental/create

- 사진(최대 10장) · 위치 검색 · 월세/매매 · 편의시설 등 **다단계 입력 폼** 구현
- 게시글(posts)과 임대 정보(rentals)를 **두 테이블에 나누어 저장하는 구조** 설계 및 구현 (문제 해결 03)



부동산 등록 — 사진·위치 입력



가격·방 타입·편의시설 선택

반복을 없애는 것보다 **무엇을 컴포넌트 밖으로 밀어낼지 정하는 것**이 어려운 문제였습니다. 화면마다 데이터도 URL 구조도 달랐지만, 그 아래에 **같은 구조**가 있었습니다.

• 사례 ① 페이지네이션 — props 3개로 5개 화면을 감당

상황 목록 화면마다 페이지 계산 · 버튼 렌더링 · 상태 관리가 각각 구현될 상황이었습니. 화면마다 다루는 데이터가 다르고(중고물품 / 채용공고 / 매물), URL 구조도 달랐습니.

판단 겉모습은 달라도 모든 목록이 공유하는 구조는 **세 가지**뿐이었습니다. 그래서 컴포넌트는 **표시만 책임지는 순수 컴포넌트**로 두고, 페이지 상태를 어디에 둘지는 **바깥의 결정**으로 밀어냈습니.

```
interface PaginationProps {
  currentPage: number;           // 지금 몇 페이지인가
  totalPages: number;           // 전체 몇 페이지인가
  onPageChange: (page: number) => void; // 바뀌면 뭘 할지 — 바깥이 결정
}
```

컴포넌트는 어떤 데이터의 목록인지, URL 규칙이 무엇인지 **모름**니다. 그래서 어느 화면에나 붙습니.

	Before	After
페이지 번호 계산	화면마다 각각 구현	컴포넌트 내부
버튼 렌더링	화면마다 각각 구현	컴포넌트 내부
페이지 상태	화면마다 각각 관리	컴포넌트 밖(URL 쿼리) — 관심사 분리
사용 화면	—	중고거래 · 구인구직 · 부동산 · 내 글 · 찜 (5개)

결과 이후 팀원이 내부 표시 규칙(중간 페이지 생략)을 교체했지만, 인터페이스가 그대로여서 5개 사용처는 한 줄도 바뀌지 않았습니. 내부 구현은 갈라끼워졌는데 경계는 살아남았습니 — 추상화 경계를 옳게 그었다는 증거였습니.

• 사례 ② 수정·삭제 버튼 — 처음부터 공통으로, 검증 로직과 함께

상황 중고물품 상세에 수정·삭제가 필요했습니. 버튼을 페이지 안에 두면, 같은 기능이 필요한 다른 상세 페이지마다 다시 만들어야 합니다.

구현 처음부터 components/common/ 에 EditButton · DeleteButton으로 분리해 만들고, 작성자 판별(로그인 사용자의 auth_id === 게시글 작성자의 auth_id)과 수정 페이지 라우팅까지 함께 구현했습니. 회원가입 트리거에서 설계한 auth_id 연결 규칙을 그대로 재사용했습니.

확산 이후 팀원이 이동 경로를 파라미터로 일반화하면서, 이 컴포넌트는 중고거래 · 구인구직 · 부동산 3개 상세 페이지의 표준이 되었습니.

재사용은 "여러 곳에서 쓰는 것"이 아니라, **무엇을 컴포넌트가 알아야 하고 무엇을 몰라도 되는지 정하는 것**이었습니다. 몰라도 되는 것(URL 규칙, 데이터 종류, 이동 경로)을 밖으로 밀어낸 만큼 컴포넌트가 오래 살아남았습니.

개발 과정에서 부딪힌 문제와, 그것을 어떻게 판단하고 해결했는지를 정리했습니다.

01. 좋아요를 누를 때마다 전체 개수를 세는 비효율

- 문제** 좋아요 수를 화면에 띄우려면 매번 좋아요 테이블 전체를 `count` 조회해야 했습니다. 목록에 카드가 늘어날수록 조회가 그만큼 반복되어 부담이 커지는 구조였습니다.
- 해결** 게시글 테이블에 `like_count` 컬럼을 두고, 좋아요 추가·삭제 시 자동으로 값을 갱신하는 **DB Trigger**를 작성했습니다. 애플리케이션이 아니라 **DB가 정합성을 책임지게** 해, 어느 경로로 좋아요가 변경돼도 수치가 어긋나지 않도록 했습니다. 클라이언트는 **낙관적 업데이트**(즉시 반영 후 실패 시 롤백)만 담당하도록 역할을 나눴습니다.
- 결과** 조회는 컬럼 하나를 읽는 것으로 끝나고, Realtime 구독과 결합해 다른 사용자의 좋아요가 새로고침 없이 즉시 반영됩니다.

02. 회원가입은 되는데, 서비스 회원 정보가 저장되지 않던 문제

- 문제** Supabase는 인증 정보를 `auth.users` 에, 서비스 프로필을 `public.users` 에 따로 보관합니다. 가입은 성공하는데 서비스 회원 정보가 생기지 않았고, 원인을 파고들자 **세 겹으로 겹쳐** 있었습니다. ① 두 테이블을 이어주는 트리거가 없었고 ② `public.users` 의 `id` 에 자동 증가 기본값이 없어 NOT NULL 위반이 났으며 ③ **트리거 함수가 에러를 던지면 `auth.users` 삽입까지 함께 롤백되어** 회원가입 자체가 실패했습니다.
- 해결** 가입 시 `public.users` 에 회원을 자동 생성하는 트리거 함수를 작성했습니다. 시퀀스로 `id` 를 채우고, `auth_id` 로 인증 계정과 서비스 프로필을 연결했으며, 중복 가입을 무시하도록 처리했습니다. 무엇보다 **함수 전체를 예외 처리로 감싸**, 프로필 생성이 실패하더라도 인증 정보는 남도록 만들어 가입이 통째로 무너지는 것을 막았습니다.
- 결과** 여기서 만든 `auth_id` 연결 규칙은 이후 관리자 회원 관리·부동산 등록·수정 삭제 권한 검증에서 회원을 식별하는 기준으로 그대로 재사용되었습니다. 인증과 서비스 데이터가 왜 분리되어 있는지를 이해한 것이 이 프로젝트에서 가장 큰 수확이었습니다.

03. 게시글과 임대 정보를 어떻게 나눠 저장할 것인가

- 문제** 부동산 글에는 제목·이미지처럼 **모든 게시글의 공통 정보**와, 보증금·방 타입·편의시설처럼 **부동산에만 있는 정보**가 섞여 있었습니다. 한 테이블에 전부 넣으면 중고거래·구인구직 글에는 빈 칸만 늘어나는 구조가 됩니다.
- 해결** 공통 정보는 `posts` , 부동산 고유 정보는 `rentals` 로 분리하고, 등록 시 `posts` 를 먼저 저장해 얻은 식별자로 `rentals` 를 저장하는 **2단계 저장 구조**를 설계했습니다.
- 결과** 게시글 종류가 늘어나도 공통 구조를 재사용할 수 있게 되었고, 게시글 삭제 시 연결된 데이터가 함께 정리되도록 처리했습니다.

세 문제 모두 "코드를 어떻게 쓸까"가 아니라 "무엇이 무엇을 책임져야 하는가"의 문제였습니다. 특히 02번의 트리거가 실패하면 회원가입 자체가 롤백된다는 사실은 에러 메시지에 드러나지 않아, 원인을 한 겹씩 벗겨내며 직접 찾아야 했습니다.

느린학습자 · 경계선 지능인을 위해 **어려운 정보를 쉬운 말로 바꿔주는 AI 서비스**입니다. 해커톤 참여작으로 **기획서 제출 후 심사를 기다리는 단계**이며, 저는 익숙한 프론트엔드 대신 **백엔드(Server)**를 자원했습니다.

기간 2026.06 ~ (심사 대기)
담당 백엔드(Server) — 대상자 분류 설계

팀 구성 5인 (프론트 · 백엔드 · AI · 기획)
기술 Next.js, Supabase, LLM API

STATUS 기획서 제출 완료 · 심사 대기 (개발 착수 전)

• 왜 백엔드를 자원했는가

파이널 프로젝트에서 Supabase를 다루며, **화면은 결국 데이터가 흐른 결과일 뿐**이라는 것을 체감했습니다. 좋아요 수 하나를 화면에 띄우는 데도 테이블 구조와 트리거를 알아야 했고, 그 경험이 없었다면 저는 그저 숫자를 받아 그리는 사람에 머물렀을 것입니다. 그래서 이번에는 **이미 잘하는 것을 반복하는 대신 모르는 쪽에 손을 들었습니다**. 첫 캡스톤에서 HTML을 모른 채 개발을 자원했던 것과 같은 선택입니다.

• 대상자 분류 알고리즘 설계

사용자의 이해 수준은 **연속적인 값**이지만, 서비스가 실제로 필요로 하는 건 "어떤 난이도의 문장을 줄 것인가" **하나**였습니다. 연속값을 그대로 다루는 대신, 복잡한 판정 문제를 **3단계 규칙으로 접었습니다**.

- **절단점수(cut-off) 기반 구간 설정** — 심리측정에서 쓰이는 기준점을 적용해 3단계 구간을 정의
- **측정표준오차(SEM)를 반영한 경계 보정** — 모든 점수에는 오차가 있으므로, 경계 부근을 단정하지 않고 오차 범위를 함께 고려
- **보수적 배정 규칙** — 애매한 구간은 더 쉬운 단계로 배정. 어려운 설명을 받아 이해하지 못하는 위험이, 쉬운 설명을 받는 불편보다 크다고 판단했기 때문

기능을 구현하기 전에 "무엇이 옳은 분류인가"를 먼저 정의해야 한다는 것을 배운 과정이었습니다. 사용자를 잘못 분류하는 일이 실제 사람의 불편으로 이어지는 서비스였기에, 코드보다 판단 기준이 먼저였습니다.

※ 기획서 제출 후 심사 결과를 기다리는 단계로, 개발 착수 전입니다. 위 내용은 실제 수행한 **설계·기획 작업**에 한해 기재했습니다.

제가 개발자가 된 지점입니다. 2023년, HTML과 CSS가 무엇인지도 모르던 상태에서 개발 담당을 자원해 만든 첫 결과물입니다. 지금 보면 투박하지만, 이 화면이 없었다면 앞의 두 프로젝트도 없었습니다.

기간 2023.09 ~ 2023.12

담당 웹 화면 개발

팀 구성 6인 (학부 캡스톤 디자인)

기술 HTML, CSS

1. 당신의 성별은 무엇입니까?

남자 : ☐ 여자 : ☐

2. 본인이 생각하는 본인의 퍼스널 컬러는 무엇입니까?

봄 원톤 : ☐ 여름 쿨톤 : ☐

가을 원톤 : ☐ 겨울 쿨톤 : ☐

3. 본인이 자주 입고, 좋아하는 옷의 색깔은 무엇입니까? (중복선택 가능)

흰색 : ☐ 회색 : ☐ 검은색 : ☐

분홍색 : ☐ 빨간색 : ☐ 주황색 : ☐

노란색 : ☐ 초록색 : ☐ 파란색 : ☐

네이비색 : ☐ 보라색 : ☐ 베이지색 : ☐

[Go for a test!](#)

사용자 설문 화면

Learn more about Personal Color

봄 원톤

여름 쿨톤

가을 원톤

겨울 쿨톤

퍼스널 컬러 유형 안내

가을 원톤에 대한 설명과 특징

가을의 컬러는 알록달록 황금빛으로 물든 우리나라 가을색을 생각하시면 좋습니다. 황토빛을 가지면서 차분하면서도 부드러운 느낌을 주는 컬러들이

가을에 속합니다. 봄 원톤과 가을 원톤의 컬러들을 구분하자면 가을 컬러가 더 짙은 황토빛이 들어서 봄 컬러들에 비해 가라앉은 느낌을 줍니다.

가을의 이미지는 따뜻하고, 부드럽고, 편안한 느낌을 줍니다. 또한 고전적이며 자연스러운 이미지를 갖고 있습니다.

가을 원톤에는 세가지 이미지가 있습니다.

어울리는 색상 & 피해야 하는 색상

유형별 상세 설명 화면

● 그때 무엇을 했는가

- 팀 회의에서 가장 모르는 역할이었던 개발을 자원 — 직접 부딪히지 않으면 아무것도 남지 않겠다고 판단
- 강의 영상을 찾아가며 HTML·CSS를 독학, 설문·유형 안내·상세 설명 화면을 구현
- 첫 팀 프로젝트로서 역할 분담과 일정 조율을 경험, 캡스톤 디자인 동상 수상

● 2023년의 화면을 그대로 싣는 이유

이 화면들은 정렬도 어긋나 있고 색도 거칩니다. 감추고 싶은 결과물이지만, 그대로 싣었습니다. 4페이지의 Kanto 화면과 이 페이지를 나란히 놓으면 2년 반 동안 제가 어디서 어디까지 왔는지 보이기 때문입니다.

당시 저는 "이게 맞나?" 하며 영상을 뒤지고, 코드를 고칠 때마다 화면이 달라지는 것을 신기해하던 사람이었습니다. 지금은 트리거로 데이터 정합성을 맡기고, 테이블을 나누는 기준을 고민하고, 컴포넌트가 무엇을 몰라도 되는지를 설계합니다. 다음 2년은 회사에서 그 거리를 만들고 싶습니다.